

Assignment 2

Name: Smruti Ranjan Behera

Roll No: 2025CA096

MCA MNNIT ALLAHABAD

Problem 1

1.1 Problem Statement

1. Given an integer array nums, sort it such that:
 1. Elements with lower frequency come first.
 2. If frequencies are equal, sort by higher numerical value first

1.2 Source Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int freq[20001];
5 int cmp(const void *a, const void *b) {
6     int x = *(int *)a;
7     int y = *(int *)b;
8
9
10    if (freq[x + 10000] != freq[y + 10000]) {
11        return freq[x + 10000] - freq[y + 10000];
12    }
13    return y - x;
14 }
15
16 void frequencySort(int *nums, int n) {
17
18     for (int i = 0; i < n; i++) {
19         freq[nums[i] + 10000]++;
20     }
21     qsort(nums, n, sizeof(int), cmp);
22 }
23
24 int main() {
25     int n ;
26     printf("\nEnter the number of elements for the array: ");
27     scanf("%d", &n);
28     int nums[n];
29     for(int i =0; i <n; i++){
30         scanf("%d", &nums[i]);
```

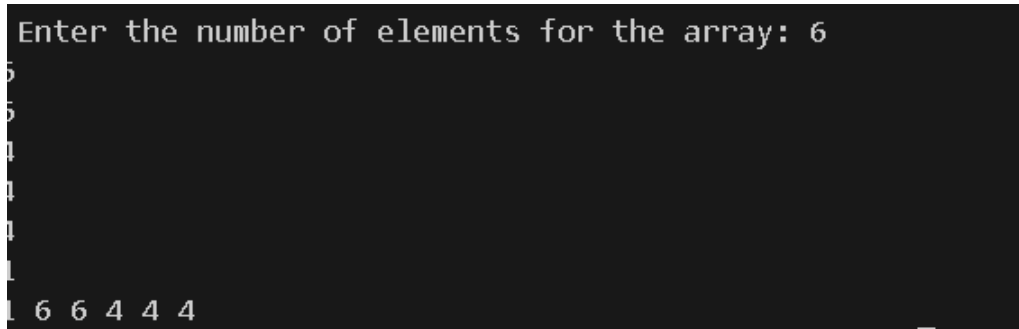
```

31     }
32
33     frequencySort(nums, n);
34
35     for (int i = 0; i < n; i++) {
36         printf("%d ", nums[i]);
37     }
38     return 0;
39 }

```

Listing 1: Source Code

1.3 Output



```

Enter the number of elements for the array: 6
6
6 6 4 4 4

```

Problem 2

2.1 Problem Statement

Given an array, count how many pairs (i, j) exist such that: $i < j$ AND $arr[i] > arr[j]$

2.2 Source Code

```

1  /*2. Given an array, count how many pairs (i, j) exist such that:
2  i < j AND arr[i] > arr[j]*/
3
4  #include <stdio.h>
5
6  int Count(int arr[], int l, int h);
7  int MergeCount(int arr[], int l, int m, int h);
8
9
10

```

```

11 int main(void) {
12     int n;
13     int arr[1000];
14
15     printf("Enter number of elements: ");
16     scanf("%d", &n);
17
18     for(int i = 0; i < n; i++) {
19         scanf("%d", &arr[i]);
20     }
21
22     int count = Count(arr, 0, n - 1);
23     printf("Inversion count = %d\n", count);
24
25     return 0;
26 }
27
28
29
30
31 int Count(int arr[], int l, int h) {
32     int count = 0;
33
34     if (l < h) {
35         int mid = l + (h - l) / 2;
36
37         count += Count(arr, l, mid);
38         count += Count(arr, mid + 1, h);
39         count += MergeCount(arr, l, mid, h);
40     }
41
42     return count;
43 }
44
45 int MergeCount(int arr[], int l, int m, int h) {
46     int i, j, k;
47     int inv_count = 0;
48     int n1 = m - l + 1;
49     int n2 = h - m;
50     int L[n1], R[n2];
51

```

```

52     for (i = 0; i < n1; i++)
53         L[i] = arr[l + i];
54     for (j = 0; j < n2; j++)
55         R[j] = arr[m + 1 + j];
56
57     i = 0;
58     j = 0;
59     k = l;
60     while (i < n1 && j < n2) {
61         if (L[i] <= R[j]) {
62             arr[k++] = L[i++];
63         } else {
64             arr[k++] = R[j++];
65             inv_count += (n1 - i);
66         }
67     }
68
69
70
71     while (i < n1)
72         arr[k++] = L[i++];
73
74     while (j < n2)
75         arr[k++] = R[j++];
76
77     return inv_count;
78 }

```

Listing 2: Source Code

2.3 Output

```

Enter number of elements: 6
1
2
5
3
6
3
Inversion count = 3

```

Problem 3

3.1 Problem Statement

An array is k -sorted if every element is at most k positions away from its sorted position. Sort the array efficiently.

3.2 Source Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Heap helpers
5 void swap(int *a, int *b) {
6     int t = *a;
7     *a = *b;
8     *b = t;
9 }
10
11 void heapify(int heap[], int n, int i) {
12     int smallest = i;
13     int l = 2*i + 1;
14     int r = 2*i + 2;
15
16     if (l < n && heap[l] < heap[smallest])
17         smallest = l;
18     if (r < n && heap[r] < heap[smallest])
19         smallest = r;
20
21     if (smallest != i) {
22         swap(&heap[i], &heap[smallest]);
23         heapify(heap, n, smallest);
24     }
25 }
26
27 int extractMin(int heap[], int *n) {
28     int root = heap[0];
29     heap[0] = heap[*n - 1];
30     (*n)--;
31     heapify(heap, *n, 0);
32     return root;
```

```

33 }
34
35 void insertHeap(int heap[], int *n, int val) {
36     int i = (*n)++;
37     heap[i] = val;
38
39     while (i && heap[(i - 1)/2] > heap[i]) {
40         swap(&heap[i], &heap[(i - 1)/2]);
41         i = (i - 1)/2;
42     }
43 }
44
45 void sortKSorted(int arr[], int n, int k) {
46     int heap[k + 1];
47     int heapSize = 0;
48
49     // Insert first k+1 elements
50     for (int i = 0; i <= k && i < n; i++)
51         insertHeap(heap, &heapSize, arr[i]);
52
53     int index = 0;
54
55     // Process remaining elements
56     for (int i = k + 1; i < n; i++) {
57         arr[index++] = extractMin(heap, &heapSize);
58         insertHeap(heap, &heapSize, arr[i]);
59     }
60
61     // Empty remaining heap
62     while (heapSize > 0)
63         arr[index++] = extractMin(heap, &heapSize);
64 }
65
66 int main() {
67     int n, k;
68
69     printf("Enter number of elements: ");
70     scanf("%d", &n);
71
72     int arr[n];
73

```

```

74     printf("Enter the elements:\n");
75     for (int i = 0; i < n; i++)
76         scanf("%d", &arr[i]);
77
78     printf("Enter value of k:");
79     scanf("%d", &k);
80
81     sortKSorted(arr, n, k);
82
83     printf("Sorted array:\n");
84     for (int i = 0; i < n; i++)
85         printf("%d ", arr[i]);
86
87     return 0;
88 }

```

Listing 3: Source Code

3.3 Output

```

Enter number of elements: 5
Enter the elements:
3
2
5
1
2
Enter value of k: 3
Sorted array:
1 2 2 3 5

```


Problem 4

4.1 Problem Statement

4. Given arrays arr1 and arr2, sort arr1 such that:

1. Elements present in arr2 appear first
2. They follow the same order as in arr2
3. Remaining elements appear at the end in ascending order

4.2 Source Code

```
1 #include <stdio.h>
2
3 int main(void) {
4     int n1, n2;
5     printf("Enter the size of two arrays:");
6     scanf("%d%d", &n1, &n2);
7
8     int arr1[n1], arr2[n2];
9     int freq[10000] = {0};
10    int result[n1];
11    int idx = 0;
12
13    printf("Enter the elements of first array:\n");
14    for (int i = 0; i < n1; i++) {
15        scanf("%d", &arr1[i]);
16        freq[arr1[i]]++;
17    }
18
19    printf("Enter the elements of second array:\n");
20    for (int i = 0; i < n2; i++) {
21        scanf("%d", &arr2[i]);
22    }
23
24    for (int i = 0; i < n2; i++) {
25        while (freq[arr2[i]] > 0) {
26            result[idx++] = arr2[i];
27            freq[arr2[i]]--;
28        }
```

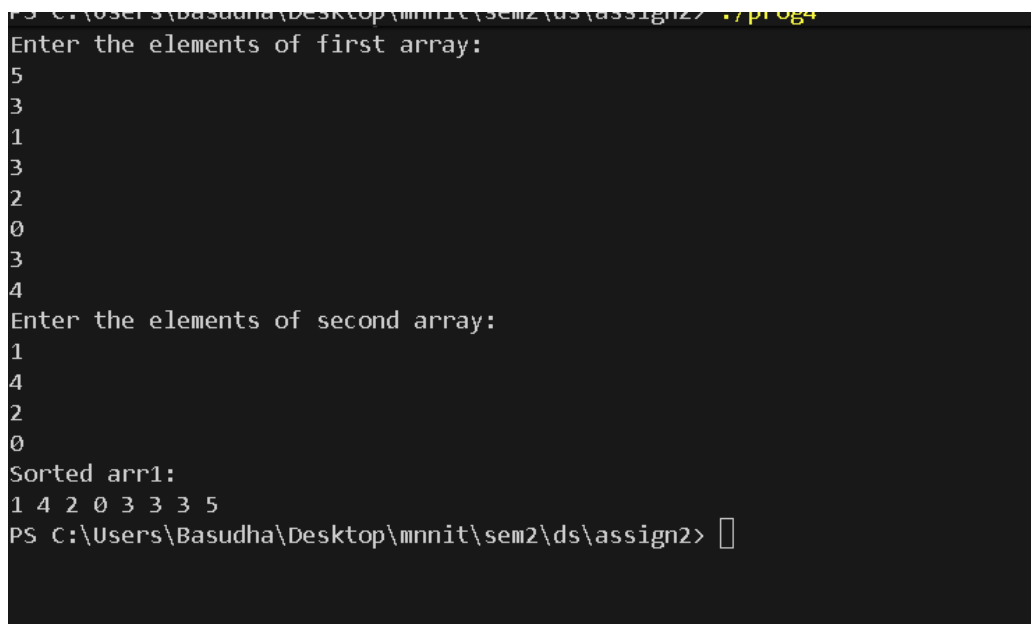
```

29     }
30     for (int i = 0; i < 10000; i++) {
31         while (freq[i] > 0) {
32             result[idx++] = i;
33             freq[i]--;
34         }
35     }
36     for (int i = 0; i < n1; i++)
37         arr1[i] = result[i];
38
39     printf("Sorted arr1:\n");
40     for (int i = 0; i < n1; i++)
41         printf("%d ", arr1[i]);
42
43     return 0;
44 }

```

Listing 4: Source Code

4.3 Output



```

PS C:\Users\Basudha\Desktop\mninit\sem2\ds\assign2> .\prog4
Enter the elements of first array:
5
3
1
3
2
0
3
4
Enter the elements of second array:
1
4
2
0
Sorted arr1:
1 4 2 0 3 3 3 5
PS C:\Users\Basudha\Desktop\mninit\sem2\ds\assign2>

```

Problem 5

5.1 Problem Statement

Given a string, sort characters in decreasing order of frequency.

5.2 Source Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef struct {
5     int freq;
6     char c;
7 } fre;
8
9 int main(void) {
10     char str[100];
11     fre f[26];
12
13     for (int i = 0; i < 26; i++) {
14         f[i].freq = 0;
15         f[i].c = 'a' + i;
16     }
17     printf("Enter a string: ");
18     fgets(str, sizeof(str), stdin);
19
20     str[strcspn(str, "\n")] = '\0';
21
22     for (int i = 0; str[i] != '\0'; i++) {
23         char ch = str[i];
24         if (ch >= 'a' && ch <= 'z') {
25             f[ch - 'a'].freq++;
26         }
27     }
28
29     for (int i = 0; i < 25; i++) {
30         for (int j = i + 1; j < 26; j++) {
31             if (f[j].freq > f[i].freq) {
32                 fre temp = f[i];
33                 f[i] = f[j];
34                 f[j] = temp;
35             }
36         }
37     }
38
39     printf("Output: ");
```

```

40     for (int i = 0; i < 26; i++) {
41         while (f[i].freq > 0) {
42             printf("%c", f[i].c);
43             f[i].freq--;
44         }
45     }
46
47     return 0;
48 }

```

Listing 5: Source Code

5.3 Output

```

PS C:\Users\Basudha\Desktop\mnit\sem2\ds\assign2> ./prog5
Enter a string: tree
Output: eert
PS C:\Users\Basudha\Desktop\mnit\sem2\ds\assign2>

```

Problem 6

6.1 Problem Statement

You are given a file containing billions of integers that cannot fit in memory. How do you sort it?

6.2 Answer:

External merge sort.

Problem 7

7.1 Problem Statement

.Given an array of integers arr of size n, write a C program to stably sort the array such that all negative numbers appear before positive numbers, while preserving the relative order of both negative and positive elements.

7.2 Source Code

```

1 #include <stdio.h>
2
3 int main() {
4     int n, arr[1000];
5     printf("Enter the number of elements: ");
6     scanf("%d", &n);
7
8     for (int i = 0; i < n; i++)
9         scanf("%d", &arr[i]);
10
11     int left = 0, right = n - 1;
12
13     while (left < right) {
14         // move left while element is negative
15         while (arr[left] < 0 && left < right)
16             left++;
17
18         // move right while element is non-negative
19         while (arr[right] >= 0 && left < right)
20             right--;
21
22         // swap misplaced elements
23         if (left < right) {
24             int temp = arr[left];
25             arr[left] = arr[right];
26             arr[right] = temp;
27         }
28     }
29
30     printf("Resulting array:\n");
31     for (int i = 0; i < n; i++)
32         printf("%d ", arr[i]);
33
34     return 0;
35 }

```

Listing 6: Source Code

7.3 Output

```
Enter the number of elements: 5
3
-2
0
1
-3
Resulting array:
-3 -2 0 1 3
PS C:\Users\Basudha\Desktop\mnit\sem2\ds\assign2> 
```

Problem 8

8.1 Problem Statement

Given an array of n integers, write a C program to determine whether the array can be sorted in non-decreasing order by performing exactly one subarray reversal.

8.2 Source Code

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int main() {
5     int n;
6     printf("Enter the number of elements: ");
7     scanf("%d", &n);
8
9     int arr[n];
10    printf("Enter the elements:\n");
11    for (int i = 0; i < n; i++)
12        scanf("%d", &arr[i]);
13
14    int l = -1, r = -1;
15
16    // Find the first decreasing point
17    for (int i = 0; i < n-1; i++) {
18        if (arr[i] > arr[i+1]) {
19            l = i;
20            break;
21        }
22    }
```

```

22     }
23
24     if (l == -1) { // already sorted
25         printf("YES\n");
26         return 0;
27     }
28
29     // Find the last decreasing point
30     for (int i = n-1; i > 0; i--) {
31         if (arr[i] < arr[i-1]) {
32             r = i;
33             break;
34         }
35     }
36
37     // Reverse arr[l..r] in place
38     int start = l, end = r;
39     while (start < end) {
40         int temp = arr[start];
41         arr[start] = arr[end];
42         arr[end] = temp;
43         start++;
44         end--;
45     }
46
47     // Check if sorted
48     bool sorted = true;
49     for (int i = 0; i < n-1; i++) {
50         if (arr[i] > arr[i+1]) {
51             sorted = false;
52             break;
53         }
54     }
55
56     if (sorted)
57         printf("YES\n");
58     else
59         printf("NO\n");
60
61     return 0;
62 }

```

8.3 Output

```
Enter the number of elements: 5
Enter the elements:
2
3
1
4
2
NO
PS C:\Users\Basudha\Desktop\mnit\sem2\ds\assign2>
assign2  Debug  C/C++ Runner: Debug Session (assign2)
```

Problem 9

9.1 Problem Statement

.Given an array of integers, write a C program to sort the array according to the absolute difference from a given value k. If two numbers have the same absolute difference, the larger number must come first

9.2 Source Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 int k; // global for comparator
6
7 int cmp(const void *a, const void *b) {
8     int x = *(int*)a;
9     int y = *(int*)b;
10    int diffX = abs(x - k);
11    int diffY = abs(y - k);
12
13    if (diffX != diffY)
14        return diffX - diffY;           // smaller absolute
        difference first
15    else
```



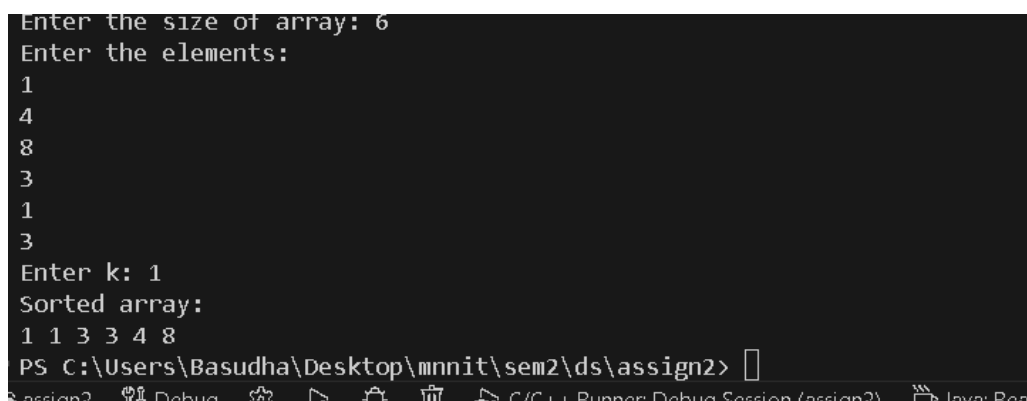
```

16         return y - x;                // larger number first if tie
17     }
18
19     int main() {
20         int n;
21         printf("Enter the size of array: ");
22         scanf("%d", &n);
23
24         int arr[n];
25         printf("Enter the elements:\n");
26         for (int i = 0; i < n; i++)
27             scanf("%d", &arr[i]);
28
29         printf("Enter k: ");
30         scanf("%d", &k);
31
32         qsort(arr, n, sizeof(int), cmp);
33
34         printf("Sorted array:\n");
35         for (int i = 0; i < n; i++)
36             printf("%d ", arr[i]);
37
38         return 0;
39     }

```

Listing 8: Source Code

9.3 Output



```

Enter the size of array: 6
Enter the elements:
1
4
8
3
1
3
Enter k: 1
Sorted array:
1 1 3 3 4 8
PS C:\Users\Basudha\Desktop\mnnit\sem2\ds\assign2>

```

Problem 10

10.1 Problem Statement

. You are given two sorted arrays A and B of sizes n and m respectively. Write a C program to merge them into a single sorted array without using any extra space.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 #include <stdio.h>
6 #include <math.h>
7
8 void mergeInPlace(int A[], int n, int B[], int m) {
9     int gap = ceil((double)(n + m) / 2);
10
11     while (gap > 0) {
12         int i, j;
13
14         // Compare elements in A
15         for (i = 0; i + gap < n; i++) {
16             if (A[i] > A[i + gap]) {
17                 int temp = A[i];
18                 A[i] = A[i + gap];
19                 A[i + gap] = temp;
20             }
21         }
22
23         // Compare elements between A and B
24         for (j = (gap > n) ? gap - n : 0; i < n && j < m; i++, j++) {
25             if (A[i] > B[j]) {
26                 int temp = A[i];
27                 A[i] = B[j];
28                 B[j] = temp;
29             }
30         }
31
32         // Compare elements in B
33         if (j < m) {
34             for (j = 0; j + gap < m; j++) {
```

```

35         if (B[j] > B[j + gap]) {
36             int temp = B[j];
37             B[j] = B[j + gap];
38             B[j + gap] = temp;
39         }
40     }
41 }
42
43     if (gap == 1)
44         gap = 0;
45     else
46         gap = ceil((double)gap / 2);
47 }
48 }
49
50 int main() {
51     int n, m;
52     printf("Enter sizes of arrays A and B: ");
53     scanf("%d%d", &n, &m);
54
55     int A[n], B[m];
56
57     printf("Enter elements of A:\n");
58     for (int i = 0; i < n; i++) scanf("%d", &A[i]);
59
60     printf("Enter elements of B:\n");
61     for (int i = 0; i < m; i++) scanf("%d", &B[i]);
62
63     mergeInPlace(A, n, B, m);
64
65     printf("After merge:\nA: ");
66     for (int i = 0; i < n; i++) printf("%d ", A[i]);
67
68     printf("\nB: ");
69     for (int i = 0; i < m; i++) printf("%d ", B[i]);
70
71     return 0;
72 }

```

Listing 9: Source Code

10.2 Output

```
Enter sizes of arrays A and B: 6 7
Enter elements of A:
1
3
6
8
9
10
Enter elements of B:
2
3
6
7
12
13
14
After merge:
A: 1 2 3 3 6 6
B: 7 8 9 10 12 13 14
```